

Online Interview Platform

Production-oriented product runbook for a local production-like runtime

Includes

- Functional scope and user journeys
- Color HLD and LLD architecture
- Technology stack and data design
- Security, reliability and AI governance
- Startup, health, backup and recovery procedures

01. Product overview

An AI-assisted technical interview platform with governed question generation, RAG grounding and auditable human review.

Implementation classification: production-oriented application code running in a local production-like environment. The included stateful topology is single-node; external production requires replicated or managed data services.

Primary users

User	What the product provides	Authorization boundary
Interviewer / Admin	Create and edit drafts, select ecosystems and technologies, generate or author mixed questions, publish, assign candidates, review answers, score and finalize results.	INTERVIEWER role plus ownership and tenant checks.
Candidate	View assigned interviews, start within the permitted window, navigate questions, autosave answers, submit, and view only released results.	CANDIDATE role plus assignment, session and release checks.

Implemented functional capabilities

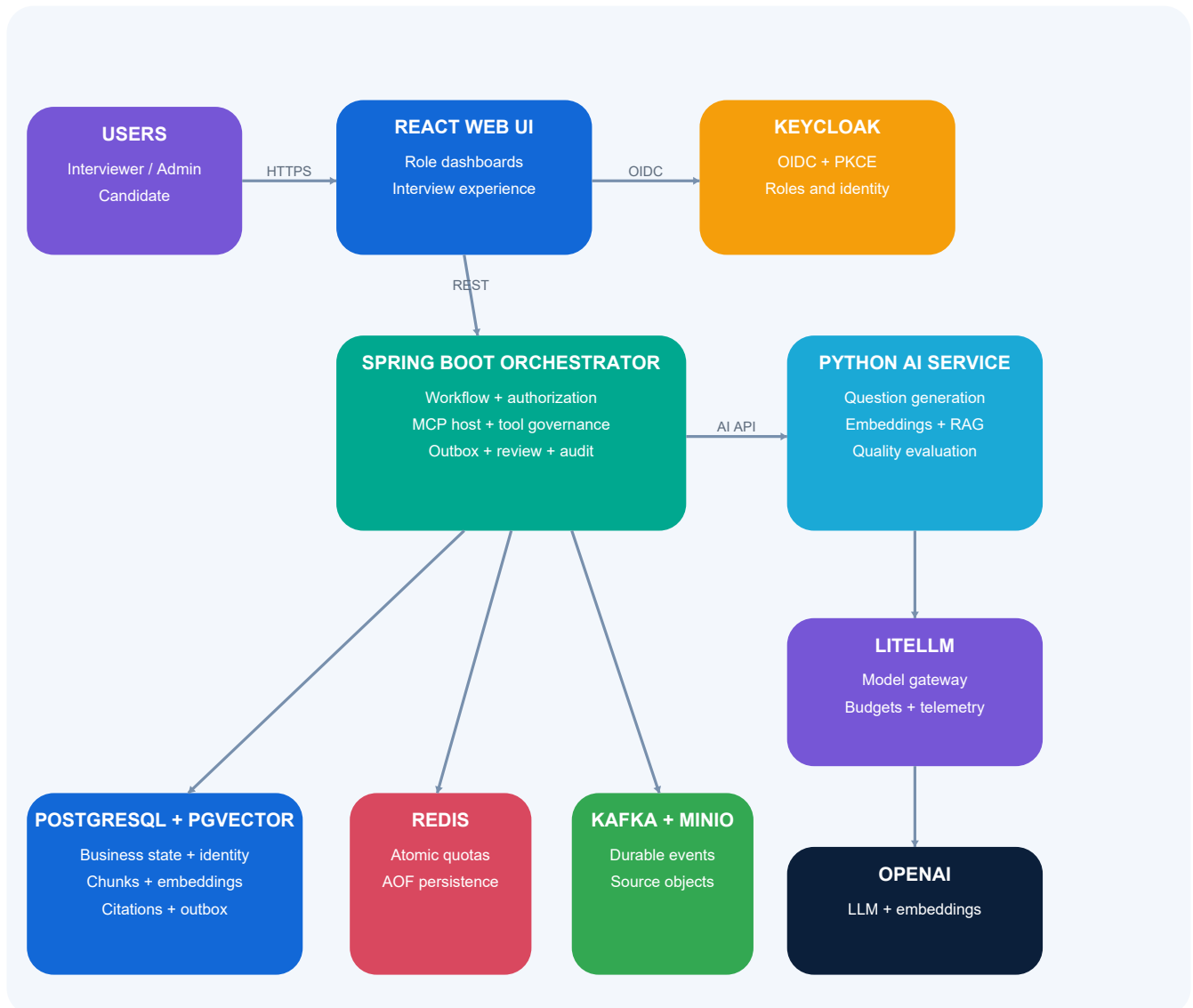
- OIDC login with Keycloak, PKCE and role-protected React routes.
- Interview draft, editable question composition, publication, candidate assignment and read-only history.
- MCQ single answer, MCQ multiple answer, short text and long text question types.
- Server-owned session lifecycle, timer, navigation, answer autosave and idempotent submission.
- Direct LLM and RAG-grounded question generation through LiteLLM model aliases.
- Knowledge collections, MinIO source objects, pgvector embeddings, filtered retrieval and citations.
- Governed MCP tools with signed scope, approval, quotas, idempotency, output scanning and audit.
- Interviewer review queue, candidate and correct answers, manual scoring, immutable outcome and audit.

Product lifecycle

Draft	Published	Assigned	In progress	Submitted	Reviewed
Editable	Frozen policy	Candidate + window	Autosave + timer	Pending review	Released result

02. High-level architecture (HLD)

Trust boundaries, synchronous calls, governed AI access and authoritative data stores.



Critical rule: the browser never calls Python AI, MCP, LiteLLM or OpenAI directly. Spring Boot authorizes every product workflow and remains the system of record.

03. HLD component responsibilities

Each component has a narrow responsibility and an explicit failure boundary.

Component	Responsibility	Data / failure behavior
React Web UI	Role-aware interviewer and candidate workspaces; navigation, timers and progress feedback.	Tokens remain in Keycloak adapter memory. The server remains authoritative.
Keycloak	OIDC identity, candidate registration and role claims.	Separate Keycloak database; no business tables.
Spring Boot orchestrator	Authorization, interview workflow, sessions, review, RAG ownership, MCP, audit and outbox.	PostgreSQL authority; optimistic locking and idempotent commands.
Python AI service	Question generation, embeddings, structured validation and quality evaluation.	Specialized AI capability; cannot authorize interview access.
LiteLLM	Stable model aliases, provider authentication, budgets, limits, fallback and AI telemetry.	Only service holding the upstream model credential.
PostgreSQL + pgvector	Business state, identity database, chunks, embeddings, citations, MCP state and outbox.	Persistent, migrated with Flyway and backed up.
Redis	Atomic cross-replica governed-tool quotas.	AOF enabled; MCP calls fail closed when unavailable.
Kafka	Durable asynchronous platform events.	Idempotent producer plus transactional outbox and bounded retry.
MinIO	Original knowledge-source objects.	Object key, byte size and SHA-256 retained in PostgreSQL.

Key request flows

Flow	Path
Business command	Browser -> Ingress -> Spring Boot -> PostgreSQL
Authentication	Browser -> Keycloak; Spring Boot validates JWT and roles
Question generation	Spring Boot -> Python AI -> LiteLLM -> OpenAI
RAG generation	MinIO source -> chunks -> embeddings -> pgvector retrieval -> grounded question + citations
Governed tool	AI host -> MCP registry/policy -> signed context -> tool handler -> domain service
Durable event	Domain transaction -> outbox row -> Kafka publisher -> topic

04. Low-level architecture (LLD)

The business backend is a modular Spring Boot application with a separately deployable AI service.



Spring Boot module contracts

Module	Owns	Important controls
Identity / profile	Identity subject, display data, role and tenant mapping.	JWT subject and tenant claim validation.
Interview	Definition, composition, passing score, publish and assignment.	Ownership, validation and optimistic versioning.
Session	Start, expiry, navigation, answers and submission.	Server time, idempotency and frozen question set.
Knowledge / RAG	Collections, documents, chunks, embeddings, retrieval and citations.	Owner filters, READY status, similarity floor and digest verification.
MCP	Registry, policies, authorization context, approval, execution and audit.	Allow-list, fail-closed quota, timeout, result scanning.
Review	Answer scoring, feedback, final result and release.	Immutable outcome and append-only audit.
Messaging	Outbox record and Kafka publication.	Database commit precedes event publication.

05. AI, RAG and MCP detailed flows

AI is a governed capability, not an authorization authority.

RAG ingestion and retrieval

Step	Processing	Guardrail
1	Interviewer submits text/Markdown source.	Role, collection ownership, media type and request-size validation.
2	Original bytes are written to MinIO.	Bounded object size; random owner-scoped key; SHA-256 calculated.
3	PostgreSQL metadata is committed.	Rollback callback removes the object when the transaction fails.
4	Content is retrieved and chunked deterministically.	Digest is verified before parsing; empty documents rejected.
5	Python creates embeddings through LiteLLM.	Stable model alias and structured response validation.
6	Vectors are stored and the document becomes READY.	Only complete READY documents are searchable.
7	Filtered cosine retrieval supplies bounded evidence.	Owner, collection, status, similarity and limit filters.
8	Question and immutable citation snapshots are stored.	Model citation IDs must be present in authorized retrieval results.

MCP governed execution

Stage	Control
Resolve	Server, tool, workflow and actor role must match an enabled allow-list policy.
Authorize	HMAC-signed, short-lived context binds actor, role, tool and target resource.
Approve	State-changing tools require resource-owner approval and an idempotency key.
Limit	Redis Lua counter atomically enforces the per-context minute quota across replicas.
Execute	Virtual-thread deadline, response-size cap and persisted execution state.
Inspect	Sensitive keys, bearer credentials, private keys and empty results are rejected.
Audit	Append-only lifecycle events retain hashes and metadata, never arguments or result bodies.

Candidate safety: hidden rubrics, expected answers and other candidates' data are never returned by candidate-scoped APIs or tools.

06. Data architecture and lifecycle

PostgreSQL is authoritative; auxiliary stores improve performance and durability without owning interview truth.

Aggregate / store	Key relationships and content	Consistency
User profile	Keycloak subject, role, tenant and status.	Unique identity subject; tenant-scoped discovery.
Interview definition	Owner, title, skills, composition, generation mode, passing score and status.	Optimistic version; publish transition.
Assignment	Interview, candidate, start/end window and attempt policy.	Authorization boundary for candidate access.
Session / answers	Frozen question set, server timestamps, autosaved content and state.	Atomic state transition; idempotent start/submit.
Review / result	Per-answer score, feedback, total, outcome and reviewer.	Immutable after finalization; append-only audit.
Knowledge	Collection, object metadata, chunks, vectors and citation snapshots.	Owner filtering and READY-only retrieval.
MCP state	Registry, policy, approvals, idempotent executions and audit.	Resource-bound signed context and database constraints.
Outbox	Aggregate, event type, payload, attempt count and publication status.	Written in domain transaction, published afterward.

Interview state machine

DRAFT	PUBLISHED / SCHEDULED	READY	IN_PROGRESS	SUBMITTED	REVIEWED
Authoring	Frozen + assigned	Window open	Autosave	Pending review	Result released

Storage ownership

Store	Authoritative content	Recovery priority
PostgreSQL	All workflow state, identity database, vectors, citations, audit and outbox.	First - restore both databases before applications.
MinIO	Original knowledge-source objects.	Second - restore before ingestion/re-ingestion.
Kafka	Asynchronous event log.	Recreate local broker; replay unpublished outbox rows.
Redis	Short-lived quota counters.	Recreate safely; governed calls remain fail-closed during outage.

07. Technology stack

Pinned or managed versions are taken from the implementation and deployment manifests.

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite 6, React Router	Role-aware SPA and interview UX.
Business backend	Java 21, Spring Boot 3.5, Spring Security, JPA	Workflow, authorization, validation and system of record.
AI backend	Python 3.12, FastAPI, Pydantic	Question generation, embeddings and AI quality.
Identity	Keycloak 26.1, OAuth/OIDC + PKCE	Authentication, registration and roles.
AI gateway	LiteLLM	OpenAI-compatible routing, model aliases, budget and telemetry.
Relational / vector	PostgreSQL 17 + pgvector 0.8.5	Transactional data and cosine vector retrieval.
Coordination	Redis 7.4 with AOF	Atomic distributed MCP quotas.
Messaging	Apache Kafka 3.9 KRaft	Durable events from transactional outbox.
Object storage	MinIO + Java SDK 8.6	Knowledge source objects and integrity metadata.
MCP	Protocol revision 2025-11-25, Streamable HTTP	Governed internal AI tools.
Observability	OpenTelemetry, Prometheus, Grafana, Loki, Tempo	Metrics, logs, traces and SLO alerts.
Delivery	Docker Compose, Kubernetes/K3d, Kustomize, Argo CD	Local runtime, packaging and GitOps.
Quality	JUnit, Mockito, JaCoCo, PIT, Vitest, pytest, Trivy	Functional, coverage, mutation and security gates.

Quality baseline

Gate	Current verified result
Java automated tests	147 tests examined by PIT; application check passed.
Line coverage	95% enforced; latest Phase 7 check passed.
Mutation testing	81% mutation score, above the configured 65% gate.
Infrastructure	Docker Compose, Kustomize overlays, Kubernetes schema and runtime dependency smoke checks.
Database	Flyway V19 successfully applied and Hibernate schema validation passed.

08. Deployment architecture

The same application images run under Docker Compose or Kubernetes; Argo CD reconciles declared Kubernetes state.

Runtime	Implemented topology	Production note
Docker Compose	PostgreSQL, Redis, Kafka, MinIO, Keycloak, LiteLLM, web UI, orchestrator, AI service and optional observability.	Suitable for local development, integration and operator drills.
Kubernetes / K3d	StatefulSets with PVCs for PostgreSQL, Redis, Kafka and MinIO; Deployments for applications; probes, resources and network policies.	Local production-like single-node topology.
Argo CD	Dev, UAT and production Applications with Kustomize overlays and migration hook.	Shared clusters require repository credentials and promotion approvals.
External production	Same stateless applications with replicated/managed data services.	Use multi-AZ PostgreSQL, Redis, Kafka and object storage plus off-site backups.

Kubernetes workload map

Workload	Kind	Persistence / scaling
postgres	StatefulSet	5 Gi PVC; authoritative; one local replica.
redis	StatefulSet	1 Gi PVC; AOF every second; one local replica.
kafka	StatefulSet	5 Gi PVC; KRaft; one local broker.
minio	StatefulSet	5 Gi PVC; one local server.
keycloak / litellm	Deployment	Separate identity database / stateless gateway.
web-ui / orchestrator / ai-service	Deployment	Resource limits, probes; production overlays support scaling.
database-migration	Argo CD hook Job	Flyway runs before application rollout.

Network and secret boundaries

- Traefik ingress exposes only approved HTTP routes; internal stores are ClusterIP/headless services.
- Default-deny network policy permits only declared application and runtime dependency flows.
- Local overlays generate development credentials; dev/UAT/prod use External Secrets.
- OpenAI credentials exist only in LiteLLM; MinIO and Redis credentials are injected from secrets.
- Runtime images are pinned by immutable digest in the Kubernetes dependency manifest.

09. Security, reliability and observability

Production readiness depends on measurable behavior during concurrency and dependency failure.

Concern	Implemented control	Operator signal
Authentication	OIDC JWT validation, role mapping and PKCE.	Keycloak readiness and authentication failures.
Authorization	Role, owner, tenant, assignment and session checks.	403/404 trends and security audit events.
Concurrency	Optimistic locking, idempotency keys and PostgreSQL row locking.	409 conflicts and duplicate execution metrics.
AI safety	Schema validation, bounded context, citation allow-list and release evaluation.	Grounding, relevance, citation and safety metrics.
Downstream failure	Bounded retry, timeout and circuit opening.	Latency, failure counters and circuit alerts.
Kafka reliability	Transactional outbox, idempotent producer and retry attempts.	Outbox pending/failed counts and broker health.
Object integrity	Size limit, random key and SHA-256 verification.	MinIO health and ingestion failures.
MCP governance	Signed scope, approval, quota, timeout, scanning and audit.	Tool outcome, quota and latency metrics.
Observability	Correlation IDs, OTLP traces, Prometheus metrics, Loki logs and Grafana.	SLO dashboard and alert rules.

Availability behavior

Dependency unavailable	Expected behavior
PostgreSQL	Readiness fails; business writes stop. Do not accept interview state without the authority store.
Redis	Governed MCP calls fail closed; core interview state remains in PostgreSQL.
Kafka	Outbox rows remain pending/failed for retry; committed business transactions remain durable.
MinIO	New document storage/ingestion fails; existing interview sessions remain usable.
LiteLLM / OpenAI	AI generation fails through bounded resilience policy; manual/frozen questions remain independent.

10. Operations runbook

Commands assume execution from the repository root.

Start the local production-like stack

Copy platform/docker/.env.example to an untracked .env, replace every placeholder, then run:

```
docker compose -f platform/docker/docker-compose.yml up -d --build
```

Optional observability profile:

```
docker compose -f platform/docker/docker-compose.yml --profile observability up -d
```

Kubernetes render and deployment

```
kubectl kustomize platform/kubernetes/overlays/local  
kubectl apply -k platform/kubernetes/overlays/local
```

Health verification

Check	Expected result
docker compose ... ps	PostgreSQL, Redis, Kafka, MinIO, AI and orchestrator healthy.
GET /actuator/health	UP with db, redis, Kafka and MinIO components.
GET /api/v1/health on AI service	HTTP 200.
kubectl get pods -n online-interview	All application pods Ready; migration Job complete.
kubectl get pvc -n online-interview	PostgreSQL, Redis, Kafka and MinIO claims Bound.

Functional smoke test

- Log in as interviewer and candidate; verify role-specific navigation.
- Create a mixed interview draft, generate/edit questions, publish and assign the candidate.
- Start as the candidate, autosave answers, navigate, submit and verify pending review.
- Review candidate and correct answers, score all items, finalize and verify result release.
- Create a knowledge collection, ingest a source, run retrieval and confirm citations.
- Inspect Prometheus/Grafana signals and ensure no secret or answer content appears in logs.

11. Backup, restore and incident actions

Persistent volumes prevent accidental pod-level data loss; they are not backups.

Create a checksum-protected backup

```
.\scripts\backup-local-runtime.ps1
```

The output directory contains online_interview.dump, keycloak.dump, knowledge-documents.tgz and SHA256SUMS. Copy it away from the cluster host.

Restore drill

```
.\scripts\restore-local-runtime.ps1 `
-BackupDirectory .\backups\online-interview-YYYYMMDD-HHMMSS `
-ConfirmRestore
```

Destructive operation: restore validates checksums, scales Keycloak and the orchestrator down, replaces both databases and mirrors the knowledge bucket, then restores application replicas.

Incident quick actions

Symptom	First checks	Safe action
Orchestrator not Ready	Pod logs; db/redis/kafka/minio health; migration Job.	Repair dependency or secret; do not bypass schema validation.
Outbox backlog	Kafka health, topic availability, outbox attempt/error fields.	Restore broker; publisher retries committed rows.
RAG ingestion failure	MinIO health, digest mismatch, AI embedding response.	Keep source metadata; retry after dependency recovery.
MCP quota failures	Redis health and credentials.	Restore Redis; never disable quota to recover traffic.
Candidate cannot start	Assignment, server time window, attempt state and JWT subject.	Correct domain data through authorized workflow.
Restore required	Confirm backup age/checksums and communicate downtime.	Use scripted restore; complete functional verification afterward.

12. Production readiness and acceptance

What is implemented, what has been verified, and what changes for an external production environment.

Area	Status	Evidence / remaining boundary
Application code	Production-oriented	Authorization, validation, audit, resilience, RAG/MCP governance and 95% coverage gate.
Local runtime	Production-like	Persistent PostgreSQL, Redis, Kafka and MinIO with probes, health, backup and restore.
CI quality	Verified	Automated tests, JaCoCo, PIT, Docker build, manifest and security workflows.
Single-node data services	Local only	Not high availability; acceptable for requested local runtime.
External production	Requires platform work	Replicated/managed data services, TLS, off-site backups, secret rotation, load/security/failover certification.
Multi-tenancy	Partial foundation	Tenant claim and selected query boundaries exist; full tenant retrofit remains a separate program.

Acceptance checklist

- All application and dependency health indicators are UP.
- Database migration is current and restart is clean.
- Interviewer and candidate end-to-end workflows pass.
- RAG objects, vectors and citations survive restart.
- Kafka outage preserves outbox events and recovery republishes them.
- Redis outage fails governed MCP calls closed.
- Backup completes, checksums validate and restore drill passes.
- No secrets, correct answers or candidate content are present in logs.
- Dashboards and alerts are exercised using controlled failures.

**Final classification: production-ready engineering foundation and production-like local runtime.
Do not describe the single-node stateful topology as highly available external production.**

Related repository references

[docs/architecture-design.md](#) | [docs/implementation-plan.md](#) | [docs/runbooks/phase-7-runtime-storage.md](#) | [platform/docker/docker-compose.yml](#) | [platform/kubernetes/base/](#)